





Pattern recognition and image analysis

Report Machine Learning labs

Paolo Alberto Bordis
Student ID: 000628838

2025/2026

Contents

1 lab 1 **1**

1.1 Dataset 1

1.2 Ridge Classifier 1

2 Lab 2 **3**

2.1 K-Nearest Neighbor and Decision Trees 3

2.1.1 K-Nearest Neighbor 3

2.1.2 Decision Tree 4

2.1.3 Random Forest 4

3 Lab 3 **5**

3.1 Multi Layer perceptron 5

4 Lab 4 **6**

4.1 Support Vector Machines (SVM) 6

4.1.1 Comparison between every classifier with this parameter tuned on test set 7

1 lab 1

1.1 Dataset

The goal of this lab is to understand the role of cross-validation for a simple classifier called Ridge Classifier. The dataset used consists of 60,000 32x32 colour images split into 10 classes. For this lab we considered only three classes: 'airplane', 'bird' and 'horse'. Instead of using the raw 8-bit RGB images, we extracted the HOG (Histogram of Oriented Gradients) descriptor, which is a feature descriptor that captures the local shape and edge structure of an image.

1.2 Ridge Classifier

The goal was to perform an analysis using this simple classifier, and in particular to investigate whether tuning one hyper-parameter was sufficient to improve its accuracy. To answer this question, we tuned the parameter α, which is the regularization term of the Ridge Classifier, and evaluated the cross-validated accuracy for different values of α to find the one that maximizes performance.

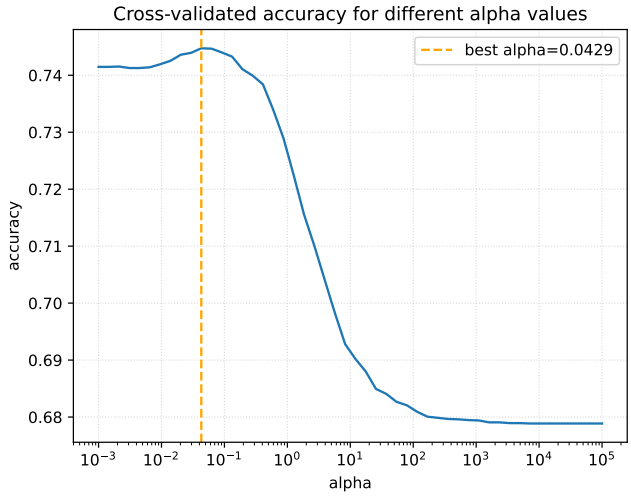


Figure 1.1: Cross-validated accuracy of the Ridge Classifier for different values of the regularization parameter α. The vertical line indicates the best α found.

As shown in Figure 1.1, the accuracy reaches a maximum of approximately more than 0.74 for α values in the range $(10^{-2}, 10^{-1})$.

Finally, we compared the default classifier ($\alpha = 1.0$) with the tuned one. The results showed a difference in accuracy, though not particularly significant.

2 Lab 2

2.1 K-Nearest Neighbor and Decision Trees

In this lab, instead of using a Ridge classifier, we evaluated the same dataset using a K-Nearest Neighbor (KNN) and a Decision Tree classifier (including an evaluation of an ensemble of tree learners).

2.1.1 K-Nearest Neighbor

K-Nearest Neighbors (KNN) is a supervised classification algorithm that predicts the class of an unseen data point based on its proximity to the training data, measured via a specific distance metric. The primary hyperparameter optimized during training is k, which defines the number of closest neighbors to consider. The final classification of the new data point is determined by a majority vote among these k nearest neighbors.

To estimate the quality of this classifier and increase its accuracy score, we tuned the parameter k, representing the number of neighbors.

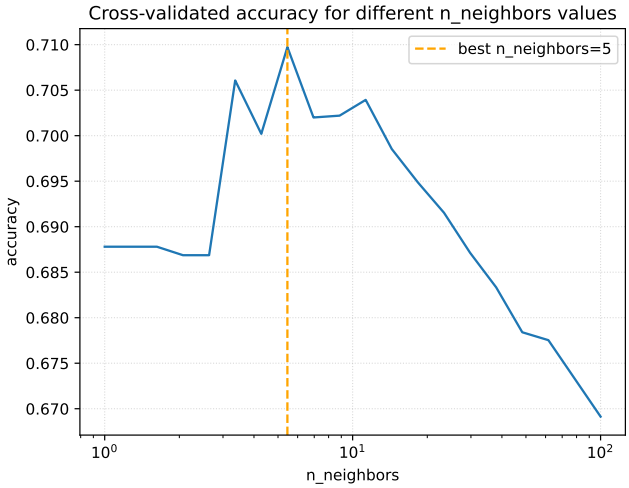


Figure 2.1: Cross-validated accuracy of the KNN classifier over the number of neighbors

2.1.2 Decision Tree

The Decision Tree Classifier is a non-parametric supervised learning algorithm that models decision boundaries through a hierarchical series of boolean rules. At each node, the algorithm partitions the feature space by selecting the specific feature and threshold that maximize information gain or minimize Gini impurity.

The most important hyperparameters that need to be tuned are the maximum depth (`max_depth`) and the minimum number of samples to split a leaf (`min_samples_leaf`). We tuned the `max_depth`, obtaining the following results:

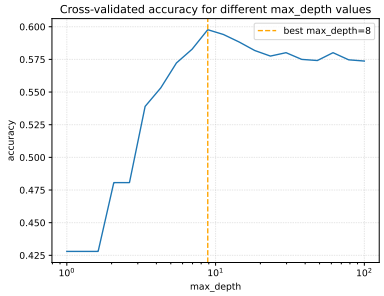


Figure 2.2: Cross-validated accuracy of the Decision Tree classifier around the maximum depth

2.1.3 Random Forest

A Random Forest is an ensemble learning method constructed from a multitude of Decision Trees. To ensure the individual trees are uncorrelated, the algorithm trains each tree on a random bootstrap sample of the training data and evaluates a random subset of features at each split. For classification tasks, the final prediction is determined by a majority vote among all the trees in the forest. This ensemble approach significantly reduces the high variance and overfitting risk inherent in a single Decision Tree.

In this case, the parameter that should be tuned is the number of trees for the classifier, which is the approach taken here.

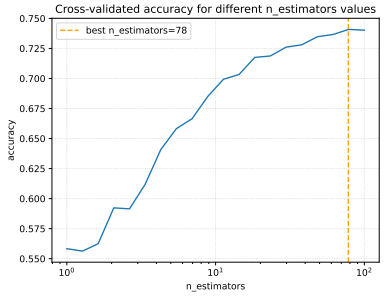


Figure 2.3: Cross-validated accuracy of the Random Forest classifier across the number of trees.

3 Lab 3

3.1 Multi Layer perceptron

Let us now introduce the Multi-Layer Perceptron (MLP) used in this lab. It is a neural network composed of nodes, called neurons, which are organized into distinct layers. It is useful to investigate whether increasing the number of neurons per layer can improve the model's robustness and accuracy score. The goal was to find a feasible architecture that achieves the maximum possible score by testing different MLP topologies, where each configuration contained a different number of hidden neurons.

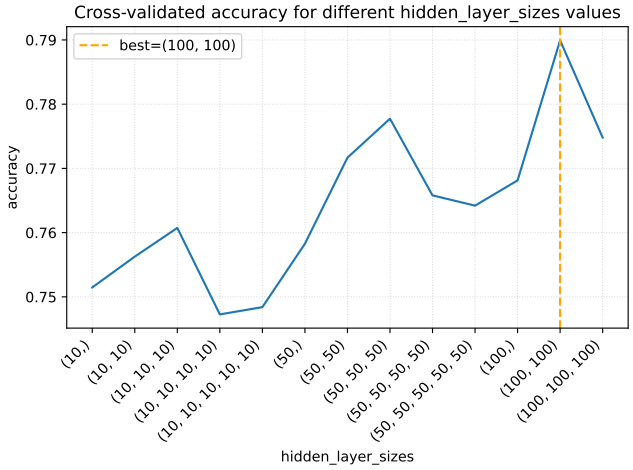


Figure 3.1: Cross-validated accuracy of MLP classifier among different architectures.

Figure 3.1 displays the different accuracies obtained using various MLP architectures. We started with a simple topology of (100,), where this notation means 1 hidden layer of 100 neurons, up to the final configuration of (100, 100, 100), which represents 3 hidden layers of 100 neurons each. Ultimately, the highest accuracy was achieved using an architecture with 2 hidden layers containing 100 neurons per layer.

4 Lab 4

4.1 Support Vector Machines (SVM)

The final model evaluated is the Support Vector Machine (SVM), a margin-based classifier that finds the optimal hyperplane to maximize class separation. Because HOG descriptors are high-dimensional and complex, we utilized a Radial Basis Function (RBF) kernel to capture non-linear patterns.

To optimize the model and prevent overfitting, we tuned the regularization parameter (C) instead doing the same for γ we only set it to scale.

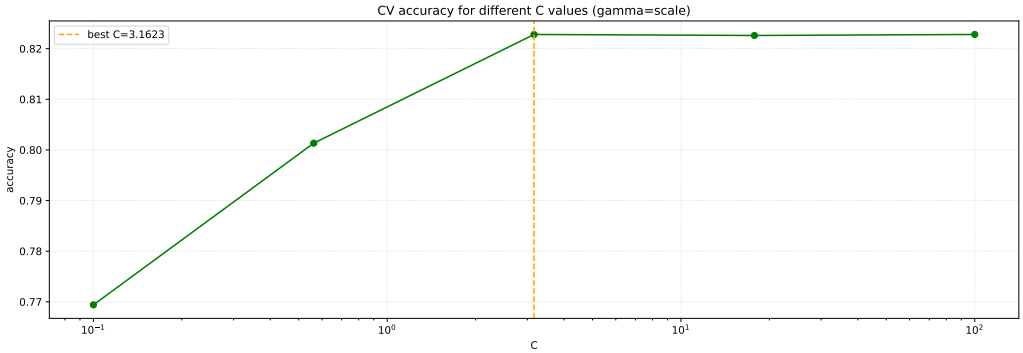


Figure 4.1: Cross-validated accuracy of the SVM classifier across different values of γ and C.

As illustrated in Figure 4.1, the selected configuration achieves a final test accuracy of approximately 0.83 with $C = 3.1623$. Compared to the default unit value, this higher C implies a lower degree of regularization. Consequently, the model enforces a stricter penalty on misclassified training samples, resulting in a narrower, "harder" decision margin that tighly fits the data distribution and relies on a well-defined subset of support vectors.

4.1.1 Comparison between every classifier with this parameter tuned on test set

Table 4.1: Comparison between every tuned classifier with some metrics to understand accuracy estimation on test set. (This table has been provieded by a method offer by the library of scikitlearn where is called classification report, and exactly provide for each class the metrics of precision, recall and F1-score)

	Precision	Recall	F1-Score	Support
Class 0	0.89	0.62	0.73	1000
Class 1	0.60	0.81	0.69	1000
Class 2	0.77	0.74	0.75	1000
Accuracy	-		0.72	3000
Macro Avg	0.75	0.72	0.72	3000
Weighted Avg	0.75	0.72	0.72	3000

(a) K-Nearest Neighbors (KNN)

	Precision	Recall	F1-Score	Support
Class 0	0.71	0.61	0.66	1000
Class 1	0.51	0.68	0.58	1000
Class 2	0.63	0.51	0.56	1000
Accuracy	-		0.60	3000
Macro Avg	0.62	0.60	0.60	3000
Weighted Avg	0.62	0.60	0.60	3000

(b) Decision Tree

	Precision	Recall	F1-Score	Support
Class 0	0.79	0.77	0.78	1000
Class 1	0.68	0.64	0.66	1000
Class 2	0.73	0.80	0.76	1000
Accuracy	-		0.73	3000
Macro Avg	0.73	0.73	0.73	3000
Weighted Avg	0.73	0.73	0.73	3000

(c) Random Forest

	Precision	Recall	F1-Score	Support
Class 0	0.82	0.85	0.83	1000
Class 1	0.75	0.73	0.74	1000
Class 2	0.83	0.82	0.83	1000
Accuracy	-		0.80	3000
Macro Avg	0.80	0.80	0.80	3000
Weighted Avg	0.80	0.80	0.80	3000

(d) Multi-Layer Perceptron (MLP)

	Precision	Recall	F1-Score	Support
Class 0	0.85	0.86	0.85	1000
Class 1	0.77	0.77	0.77	1000
Class 2	0.86	0.84	0.85	1000
Accuracy	-		0.83	3000
Macro Avg	0.83	0.83	0.83	3000
Weighted Avg	0.83	0.83	0.83	3000

(e) Support Vector Machine (SVM)

As observed in Table 4.1, while the SVM classifier does not achieve the absolute highest score in every single isolated metric (e.g., KNN yields a higher precision for Class 0), it delivers the best overall performance due to its remarkable stability across all classes. Therefore, it is the preferable model for this task, achieving not only the highest overall accuracy (0.83) but also robust performance on a per-class basis. While other classifiers might show marginal improvements in predicting a specific class, the SVM remains the superior choice overall because it maintains a highly consistent balance between precision, recall, and F1-score.